

# Building a universal game-playing agent infrastructure to optimize performance

Maria Manina  
IM updates

10.03.2026

# Research question: reminder

**Goal:** create a general dialogue game-playing agent that

- does not rely on game-specific solvers
- does not modify game-specific rules
- must adhere to the standard clembench game flow

**Method:** adding different modular components around vanilla LLMs

# Research question: reminder

**Goal:** create a general dialogue game-playing agent that

- does not rely on game-specific solvers
- does not modify game-specific rules
- must adhere to the standard clembench game flow

**Method:** adding different modular components around vanilla LLMs

**Previously:**

- Tested a LangChain agent architecture with tools for remembering game rules and observing gameplay
- Yielded an improvement in % played, but a decrease in clemscore/quality score

# Testing

- Selected 3 clembench games with diverse characteristics (single-player vs. multiplayer, word-based or exploration-based): **taboo**, **referencegame**, **textmapworld**
- Newly **generated instances** (v3.0), 15-25 instances per game
- Conducted manual transcript analysis after gameplay

# Baseline revisited

Switched to a simpler baseline for fairer evaluation: A basic ClemAgent (Qwen3-VL-30B-A3B-Instruct-FP8)

| <b>clemscore</b> | <b>all, avg<br/>%<br/>Played</b> | <b>all, avg<br/>Quality<br/>Score</b> | <b>reference<br/>-game, %<br/>Played</b> | <b>reference-<br/>game,<br/>Quality<br/>Score</b> | <b>taboo, %<br/>Played</b> | <b>taboo,<br/>Quality<br/>Score</b> | <b>textmapworld<br/>% Played</b> | <b>textmapworld<br/>Quality Score</b> |
|------------------|----------------------------------|---------------------------------------|------------------------------------------|---------------------------------------------------|----------------------------|-------------------------------------|----------------------------------|---------------------------------------|
| <b>18.28</b>     | <b>67.35</b>                     | <b>27.14</b>                          | <b>100.0</b>                             | <b>60 (std 50)</b>                                | <b>100</b>                 | <b>21,43<br/>(std 42,58)</b>        | <b>2.04</b>                      | <b>0.0</b>                            |

# Agent architectures tested

- LangChain agents with different tool sets for game rules adherence and strategy
- ClemAgent + simple planning loop

# LangChain agent + core tools

- Refined prompts and tool documentation since the previous iteration
- If human gameplay is the gold standard, we might want to encourage adherence to **game rules** and **strategy**
- Goal: address the self-repetition problem (to put it in a fancy way, encourage **inductive reasoning**)

# LangChain agent + core tools

**Core tools** (always on, no forced use):

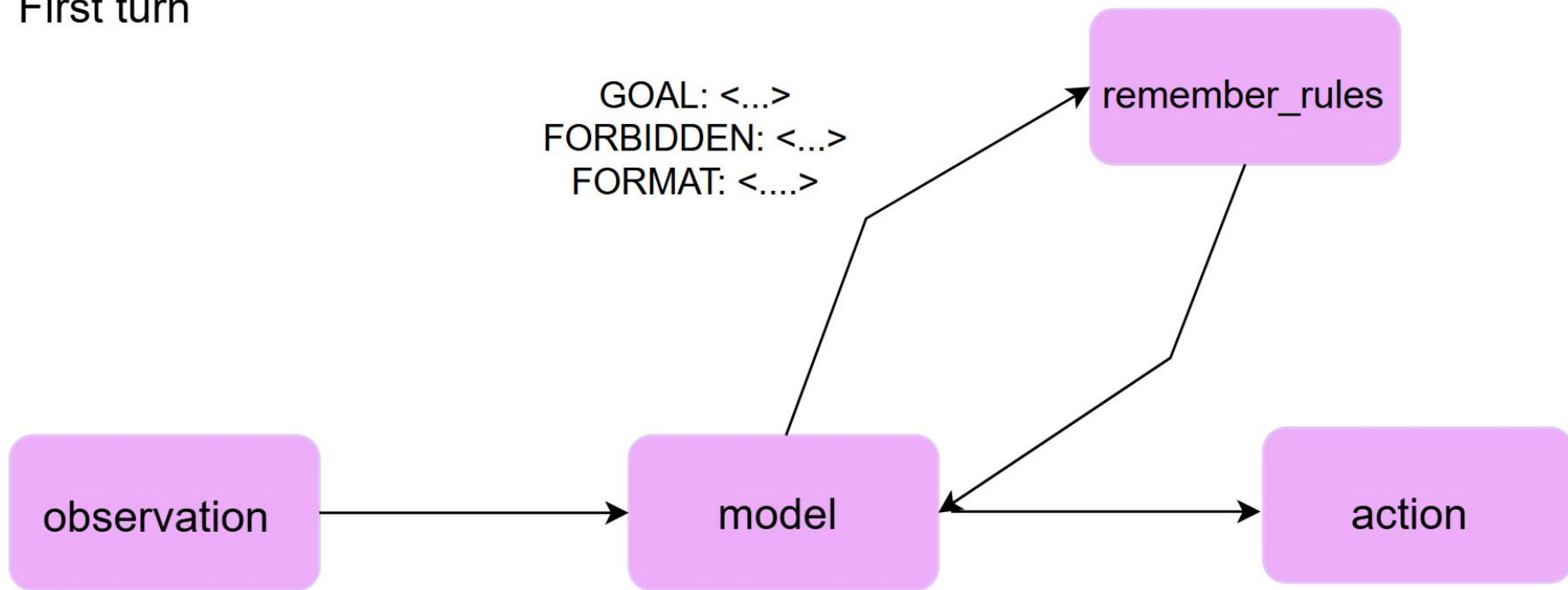
- `observe_game` (store important game flow information for later — visited places, used guesses etc., and reason about the next steps)
- `get_game_observations` (retrieve stored information)
- `remember_rules` (stores the games rules and the output format)
- `recall_rules` (retrieve the stored information about the rules)

A **short-term memory** component (stores the `ClemObservations` together with tool calls and their results)

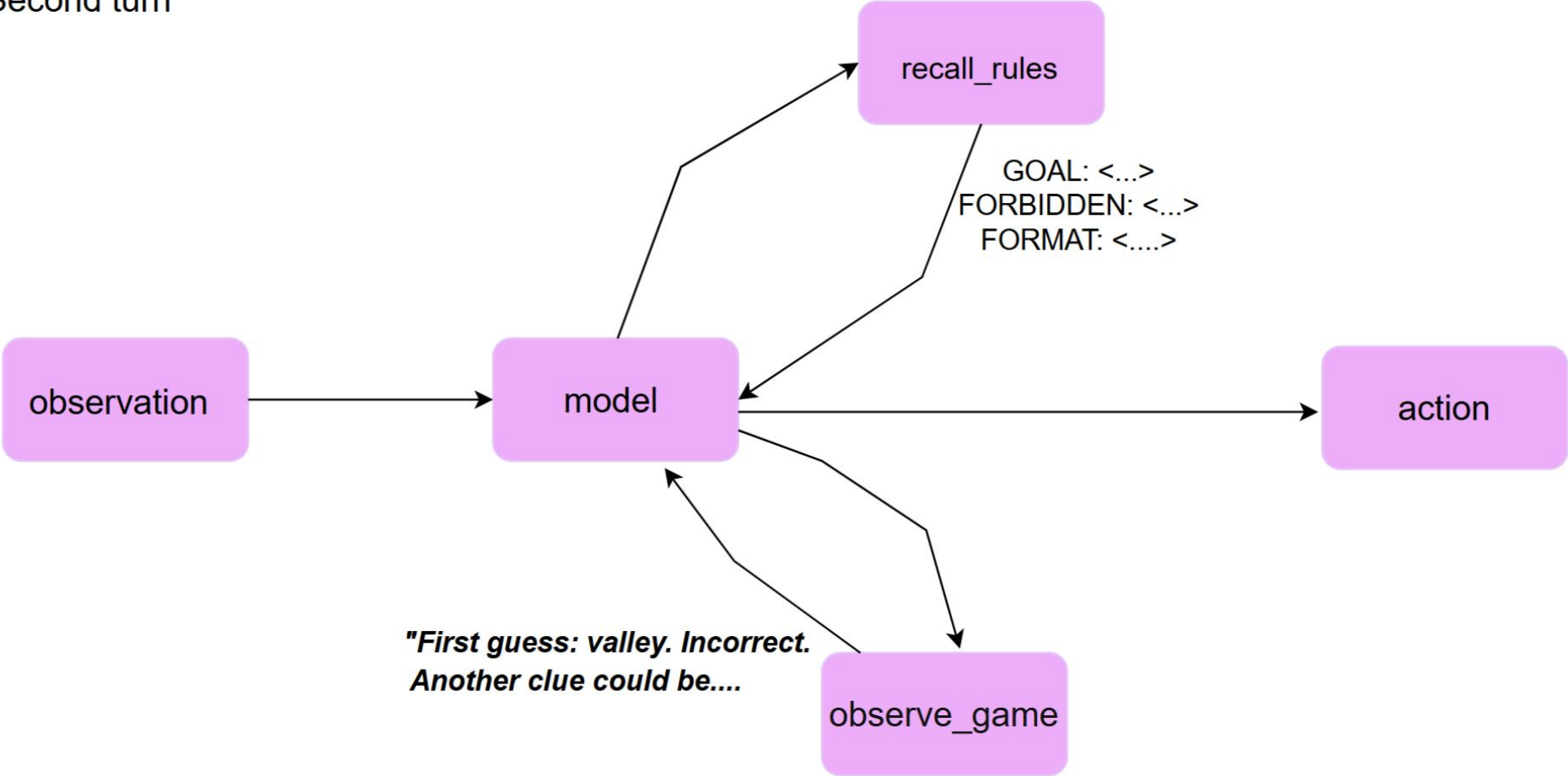


# LangChain agent + core tools

First turn



Second turn



# LangChain agent + core tools: results

| <b>clmscore</b> | <b>all, avg<br/>%<br/>Played</b> | <b>all, avg<br/>Quality<br/>Score</b> | <b>reference-<br/>game, %<br/>Played</b> | <b>reference-<br/>game,<br/>Quality<br/>Score</b> | <b>taboo, %<br/>Played</b> | <b>taboo,<br/>Quality<br/>Score</b> | <b>textmapworld,<br/>% Played</b> | <b>textmapworl<br/>d, Quality<br/>Score</b> |
|-----------------|----------------------------------|---------------------------------------|------------------------------------------|---------------------------------------------------|----------------------------|-------------------------------------|-----------------------------------|---------------------------------------------|
| <b>37,83</b>    | <b>74</b>                        | <b>51,12</b>                          | <b>92</b>                                | <b>69,57 (std<br/>47,05)</b>                      | <b>100</b>                 | <b>20<br/>(std 36,84)</b>           | <b>30</b>                         | <b>63,8<br/>(std 13.0)</b>                  |

- `remember_rules()` and `recall_rules()` are used constantly and correctly, but create unnecessary doubling with the short-term memory
- `observe_game()`: mostly incorrect usage (repeating the GM's messages)
- `get_observations()` never got used

## LangChain agent + core tools (redundant version)

- Rule tools may not be necessary, but **observe\_game** and **get\_observations** appear important for grounding (especially in multi-turn games)
- **No major performance changes overall** (however, still performs better than the baseline)
- **textmapworld:** percentage of games played increased from 30% → 78%, quality score stayed the same

# LangChain agent + core tools (redundant version)

- An example of `observe_game()` used:

```
1. Started in Guest room with only west direction available\n2. Moved to Exercise room with directions  
north and east available\n3. Moved to Study room with directions north and south available\n4. Moved to  
Wine cellar with only south direction available\n5. Returned to Study room with directions north and  
south available
```

```
"Main Score": 66.6, "efficiency": 50.0, "exploration": 100.0
```

# Long-term Planning LangChain agent

- Two tools: `write_trajectory()` and `get_trajectory()` (contribute to the long-term storage)
- The short-term memory component remains

# Long-term Planning LangChain agent:results

| <b>clemscore</b> | <b>all, avg<br/>%<br/>Played</b> | <b>all, avg<br/>Quality<br/>Score</b> | <b>reference<br/>-game, %<br/>Played</b> | <b>reference-<br/>game,<br/>Quality<br/>Score</b> | <b>taboo, %<br/>Played</b> | <b>taboo,<br/>Quality<br/>Score</b> | <b>textmapworld,<br/>% Played</b> | <b>textmapworl<br/>d, Quality<br/>Score</b> |
|------------------|----------------------------------|---------------------------------------|------------------------------------------|---------------------------------------------------|----------------------------|-------------------------------------|-----------------------------------|---------------------------------------------|
| <b>33,14</b>     | <b>94</b>                        | <b>35,26</b>                          | <b>100</b>                               | <b>40 (std 50)</b>                                | <b>100</b>                 | <b>17,78<br/>(std 35,34)</b>        | <b>82</b>                         | <b>47,99<br/>(std 21,76)</b>                |

# Occasional signs of reasoning?

```
"observations": [
```

```
    "First clue: 'Without any deviation or error.' This suggests a word related to precision, accuracy, or perfection. Possible words: 'exact', 'perfect', 'accurate', 'precise', 'flawless'." ,
```

```
    "Second clue: 'With no room for error.' This reinforces the theme of precision and accuracy. 'Exact' was guessed, but the clue is still consistent with words like 'precise', 'accurate', 'perfect', or 'flawless'. Need to consider words that emphasize strictness or lack of tolerance for mistakes." ,
```

```
    "Third clue: 'In a way that matches perfectly.' This strongly suggests a word meaning 'exact' or 'perfect'. 'Precise' was guessed, but the clue aligns well with 'exact', 'perfect', or 'accurate'. Given the progression, 'perfect' seems highly likely as it fits all clues: no deviation, no room for error, and matching perfectly." ,
```

```
<...> ]
```



# The problem of uncontrolled variables

My goal is to create a controlled experimental environment, but several factors introduce **variability**:

- the clembench setup (corrections, instance generators written by different contributors)
- model selection (not always optimal due to cost constraints)
- Last but not least: agent architecture itself (e.g., the LangChain agent, where tool use is controlled by the framework)

**Open question:** where do the observed agent mistakes actually originate?

# From complex structures to controlled variables

- I'm planning to conduct more experiments by shifting from complex structured architectures to simple, modular add-ons to the ClemAgent class
- E.g. A default ClemAgent with an additional ACT: loop

*What would be the best next action? Reply only with "ACT:" followed by your action.*

| agent                        | clmscore     | referencegame, % Played | reference game, Quality Score | taboo, % Played | taboo, Quality Score | textmapworld, % Played | textmapworld, Quality Score |
|------------------------------|--------------|-------------------------|-------------------------------|-----------------|----------------------|------------------------|-----------------------------|
| baseline_ClemAgent           | 18.28        | 100                     | 60                            | 100             | 21.43                | 2.04                   | 0.00                        |
| ClemAgent_act_loop           | 19.86        | 4.17                    | 100                           | 100             | <b>25</b>            | 38.78                  | 0.00                        |
| LangChain_4_tools            | <b>37.83</b> | 92                      | 100                           | 100             | 20                   | 30                     | <b>63.8</b>                 |
| LangChain_planning_2tools    | 33.14        | 100                     | 40                            | 100             | 17.78                | <b>82</b>              | 47.99                       |
| LangChain_tag_extractor_tool | 30.30        | 100                     | 80                            | 0.00            | NaN                  | 28                     | 62                          |

# Results and future plans

- In **LangChain**, the problem of agents not using tools despite explicit instructions remained unresolved
- **LangChain constructor notebook** (technical basis for other end-to-end compatible experiments) created
- More controlled simple experiments to follow
- Extrapolation is the main goal